



EXPRESIONES LAMBDA

¿QUÉ ES?

Una expresión lambda es una función anónima, es decir, una función sin un nombre, que se puede utilizar para crear delegados o árboles de expresión en lugar de métodos tradicionales. En C#, se introdujo en la versión 3.0 y desde entonces ha sido una herramienta poderosa para escribir código más conciso y legible.

SINTAXIS

La sintaxis básica de una expresión lambda en C# es:

(parameters) => expression;

Por ejemplo:

(int x, int y) => x + y;

CARACTERÍSTICAS

Función anónima: No tiene un nombre.

Sintaxis concisa: Es más corta y más fácil de leer en comparación con los métodos tradicionales.

Se puede usar donde se espera un delegado o un árbol de expresión.

USO EN UNITY

En Unity, las expresiones lambda se utilizan comúnmente para trabajar con eventos y delegados. Permiten escribir código de una manera más limpia y directa, especialmente cuando se trata de suscribir y desuscribir eventos.

Por ejemplo :

```
GetComponent<Button>().onClick.AddListener(() =>  
    ChangeColor(Color.red));
```


USO EN UNITY

// Definimos una expresión lambda que evalúa si un número es mayor que 10

```
Func<int, bool> mayorQueDiez = (num) => num > 10;
```

```
Debug.Log(mayorQueDiez(5)); // False
```

```
Debug.Log (mayorQueDiez(15)); // True
```

Func<int, bool>: Func es un delegado genérico proporcionado por .NET que representa una función que toma un parámetro de tipo int y devuelve un valor de tipo bool.

El primer tipo genérico (int) es el tipo del parámetro que la función recibe.

El segundo tipo genérico (bool) es el tipo del valor que la función devuelve.

mayorQueDiez: Es el nombre de la variable que contiene la expresión lambda.

(num) => num > 10: Es la expresión lambda que define la función:

(num): Es el parámetro de tipo int que la función recibe. Este es el número que se evaluará.

num > 10: Es la condición que se evalúa. La expresión devuelve true si num es mayor que 10, y false en caso contrario.

CONCLUSIÓN

Las expresiones lambda en C# son una poderosa herramienta para escribir código más legible y mantenible al encapsular lógica de condicionales de manera concisa y directa. Su flexibilidad y claridad las hacen ideales para mejorar la productividad del desarrollador al escribir funciones anónimas en el contexto donde se necesitan.